n8n 노코드 멀티 에이전트 과정 · 3장

실행 환경과 데이터 흐름 읽기

설계를 실행되는 워크플로우로 옮긴다. 실행 환경을 준비하고,
노드 사이를 흐르는 데이터를 읽고, 필요한 기능을 찾아 연결한다

n8n 실행 환경 준비부터 데이터 흐름 읽기와 기능 탐색까지

학습 목표

이 장을 마치면 할 수 있는 것

1. 컨테이너로 실행, 직접 설치, 서비스 이용의 세 가지 **실행 방식**을 업무 조건으로 비교하고 고른 이유를 설명한다.
2. 실행 환경에서 **지켜야 할 것**과 그 저장 위치를 설명하고, n8n이 자기 정보를 저장하는 데이터베이스와 업무 데이터를 저장하는 데이터베이스를 구분한다.
3. 노드 사이를 흐르는 **아이템**의 구조를 설명하고, 어느 노드에서 아이템 수가 바뀌는지 예상한다.
4. **표현식**으로 필드를 참조하고, 값이 없거나 데이터 타입이 다른 값을 계약에 맞게 바꾼다.
5. **실행 결과**를 읽어 값이 처음 틀어진 지점을 찾는다.
6. 업무 요구를 **검색 문장**으로 바꾸어 공식 문서에서 기능을 찾고, 작은 데이터로 확인한 뒤 연결한다.

이 장의 실습에서는 실행 환경, 문의를 계약 형식으로 맞추고 경로를 나누는 최소 워크플로우, 기능 탐색 기록을 만든다.

이 장에서 다루는 문제

설계는 끝났지만 아직 아무것도 실행되지 않는다

1장에서는 업무를 단계로 나누고 단계마다 결과를 누가 정할지 골랐다. 2장에서는 데이터 계약, 상태, 실패 경로, 인수 조건을 설계했다. 이제 설계를 실행되는 워크플로우로 옮기려 하면, 설계할 때 생각하지 않아도 되던 질문이 한꺼번에 생긴다.

저장

만든 워크플로우는 어디에 저장되는가? 컴퓨터를 껐다 켜면 사라지는가?

시각

"지금 시각"은 어느 나라의 시각인가?

빈 값

분명히 값을 넣었는데 다음 단계에서 빈 값이 나오면 어디를 봐야 하는가?

이 장은 이런 질문을 세 가지로 묶어 답한다.

이 장에서 다루는 문제

이 장에서 답할 세 가지 질문

질문	다루는 내용	다루는 절
이 설계를 어디에서 실행하는가?	실행 방식의 선택과 환경이 지켜야 할 것	3.1, 3.2
실행 중에 데이터는 어떤 모습으로 흐르는가?	아이템, 필드 참조와 표현식, 실행 결과 읽기	3.3, 3.4, 3.5
필요한 기능을 어떻게 찾아 연결하는가?	요구에서 기능을 찾는 절차	3.6

세 질문은 순서대로 이어진다. 환경이 있어야 실행할 수 있고, 데이터가 어떤 모습으로 흐르는지 알아야 결과를 읽을 수 있고, 결과를 읽을 수 있어야 새 기능을 찾아 붙여 보고 맞는지 판단할 수 있다.

3.1 n8n을 어디에서 실행할 것인가

n8n을 실행하는 세 가지 방식

n8n을 쓰려면 먼저 설치해야 한다. 설치 방법은 크게 세 가지 방식으로 나뉜다

컨테이너로 실행(Docker)

n8n과 실행에 필요한 것들을 하나로 묶은 **Docker 이미지**를 내려받아 **컨테이너**로 실행한다.

직접 설치

n8n이 필요로 하는 실행 환경(런타임)인 **Node.js**를 n8n을 실행할 컴퓨터나 서버에 먼저 깔고, 그 위에 npm으로 n8n 패키지를 설치한다.

서비스 이용(n8n Cloud)

n8n을 만든 쪽이 서버를 운영해 준다. 설치할 것이 없고 웹 주소로 접속해 바로 쓴다.

3.1 n8n을 어디에서 실행할 것인가

세 가지 실행 방식 비교

셋 중 무엇이 옳다고 정해져 있지 않다

구분	컨테이너로 실행	직접 설치	n8n Cloud
장점	환경이 같게 재현된다. 버전을 정확히 고정하고, 데이터베이스 같은 서비스를 구성 파일 하나로 함께 실행한다	Docker 없이 Node.js만 있으면 바로 시작한다	설치와 서버 운영 없이 웹 주소로 접속해 바로 쓴다
단점	Docker를 설치하고 기본 개념을 익혀야 한다. 서버를 운영하면 서버 비용이 든다	Node.js 버전이나 다른 프로젝트와 충돌할 수 있고, 업그레이드를 직접 한다	요금제에 따른 이용료가 든다. 버전과 서비스 밖의 연결을 운영자가 정하기 어렵다
운영자가 관리하는 것	컨테이너 실행 환경과 구성 파일	Node.js 버전, 전역 패키지, n8n 업그레이드	없다
버전 고정	이미지 태그로 정확히 고정한다	설치 명령으로 지정하되 Node.js 버전의 영향을 받는다	서비스가 올리는 버전을 따른다
다른 프로그램과의 충돌	컨테이너 안에 갇히므로 영향이 적다	같은 Node.js를 쓰는 다른 프로젝트와 충돌할 수 있다	해당 없다

이 교재의 실습은 컨테이너로 실행하는 방식으로 진행한다.

3.2 실행 환경이 지켜야 하는 것

컨테이너 바깥에 두는 두 가지

컨테이너를 지우면 안의 내용도 함께 사라지므로, 보관할 데이터와 설정은 바깥에 두고 컨테이너에 연결해 쓴다

구분	볼륨(volume)	환경 변수(environment variable)
무엇인가	Docker가 컨테이너에 연결해 주는 바깥의 저장 공간. 컨테이너는 볼륨을 자기 폴더처럼 쓴다	컨테이너를 만들 때 Docker가 바깥에서 넣어 주는 설정 값
무엇을 저장하는가	n8n이 실행되며 쌓는 정보: n8n 설정 파일, 데이터베이스 파일(워크플로우, 자격 증명, 실행 기록, 업무 데이터)	n8n과 데이터베이스가 실행될 때 읽는 설정 값: 데이터베이스 비밀번호, 암호화 키, 시간대
왜 바깥에 두는가	컨테이너 안에 그냥 두면 컨테이너를 지울 때 함께 사라진다	컨테이너는 이미지에서 그대로 만들어져 누가 실행하든 내용이 같다. 쓰는 사람마다 달라지는 값은 이미지 안에 넣을 수 없다
누가 언제 값을 기록하는가	컨테이너가 실행되는 동안 n8n과 데이터베이스가 데이터를 기록한다	운영자가 컨테이너를 만들기 전에 파일에 값을 적어 두고, 컨테이너를 만들 때 전달한다

환경 변수는 compose.yml과 같은 폴더의 .env 파일에 적는다. Docker Compose가 컨테이너를 만들 때 compose.yml의 ${이름}을 .env의 값으로 채운다.

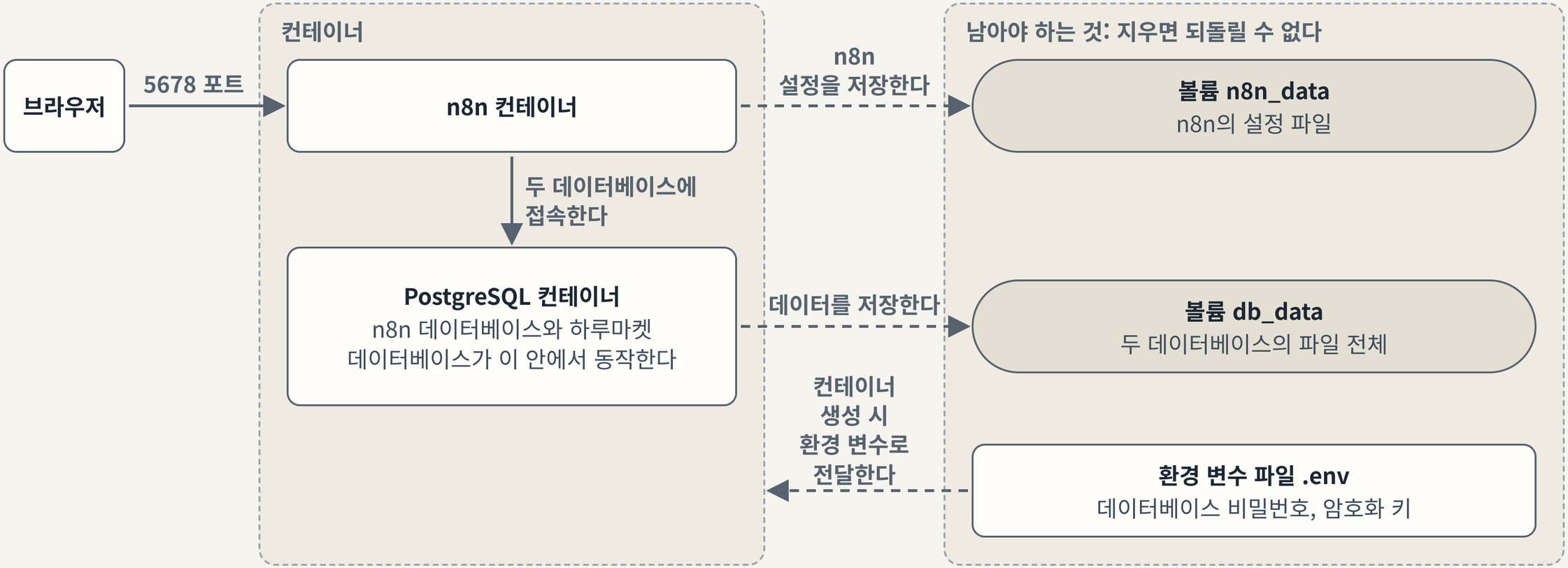
```
# .env (비밀번호와 암호화 키가 있으므로 공유하지 않는다)
POSTGRES_PASSWORD=직접-정한-비밀번호
TIMEZONE=Asia/Seoul
```

```
# compose.yml
  POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
  TZ: ${TIMEZONE}
```

3.2 실행 환경이 지켜야 하는 것

실습 환경의 구성

지워도 되는 것과 남아야 하는 것으로 나누어 본다. 양 끝이 둥근 상자가 볼륨이다



3.2 실행 환경이 지켜야 하는 것

실습 환경 그림에서 읽어야 할 것

컨테이너

1. **n8n 컨테이너**: n8n을 실행하는 환경이다. 브라우저에서 5678 포트로 접속해 워크플로우를 만들고 실행한다.
2. **PostgreSQL 컨테이너**: PostgreSQL 데이터베이스를 실행하는 환경이다. 이 안에 데이터베이스 두 개를 만든다. 하나는 n8n이 워크플로우, 자격 증명, 실행 기록 같은 자기 데이터를 저장하는 n8n 데이터베이스이고, 다른 하나는 하루마켓의 문의 기록과 요청 기록을 저장하는 하루마켓 업무 데이터베이스다.

볼륨

1. **n8n 컨테이너에 연결한 볼륨 n8n_data**: n8n의 설정 파일을 저장한다.
2. **PostgreSQL 컨테이너에 연결한 볼륨 db_data**: 두 데이터베이스의 파일 전체를 저장한다. 컨테이너를 지워도 같은 볼륨으로 다시 만들면 내용이 그대로 돌아온다.

컨테이너 둘은 다시 만들 수 있지만, 볼륨 둘과 환경 변수 파일을 유지하면 이전 환경을 그대로 사용할 수 있다.

3.2 실행 환경이 지켜야 하는 것

내부 저장 데이터베이스와 업무 데이터베이스

n8n이 관리하는 정보와 업무가 관리하는 정보를 따로 저장한다

내부 저장 데이터베이스: n8n이 자기가 관리하는 정보(워크플로우의 구성, 자격 증명, 지난 실행의 기록)를 저장하는 곳. 별도 설정이 없으면 파일 하나에, 설정을 바꾸면 PostgreSQL 같은 데이터베이스에 저장한다. (기본은 sqlite3 을 이용한다.)

업무 데이터베이스: 2장에서 설계한 문의 기록과 요청 기록처럼 업무가 관리하는 정보를 저장하는 곳.

구분	내부 저장 데이터베이스	업무 데이터베이스
무엇을 저장하는가	워크플로우, 자격 증명, 실행 기록	업무의 기록. 예를 들어 문의 기록과 요청 기록
누가 읽고 쓰는가	n8n이 직접	워크플로우가 노드를 통해
구조를 누가 정하는가	n8n	설계자. 2장의 데이터 계약이 그 설계다
n8n을 다른 도구로 바꾸면	함께 사라지거나 옮겨야 한다	그대로 남는다
업무 상태 조회의 근거인가	아니다	그렇다

3.2 실행 환경이 지켜야 하는 것

실행 환경에서 지켜야 할 다섯 가지

환경을 새로 만들거나 옮길 때 이 다섯 가지를 확인한다. 하나라도 빠지면 오른쪽 열의 문제가 생긴다

점검 항목	설정 대상	역할	없거나 틀리면 생기는 일
영속 저장	볼륨(n8n_data, db_data)	컨테이너를 다시 만들어도 워크플로우와 설정을 남긴다	컨테이너를 다시 만들 때 모두 사라진다
데이터베이스	n8n 데이터베이스와 업무 데이터베이스(PostgreSQL)	워크플로우, 자격 증명, 실행 기록, 업무 데이터를 저장한다	저장할 곳이 없거나, 업무 데이터와 도구의 데이터가 뒤섞인다
암호화 키	.env의 N8N_ENCRYPTION_KEY	저장한 자격 증명을 암호화하고 다시 읽게 한다	자격 증명을 열 수 없어 연결을 처음부터 다시 만들어야 한다
시간대	.env의 시간대 값(일정 트리거와 컨테이너 시스템)	시각 계산과 정해진 시각의 실행에 기준을 준다	정해진 시각의 실행과 기한 계산이 어긋난다
자격 증명	n8n에 등록한 자격 증명(Credentials)	연결 정보를 워크플로우 밖에 따로 저장한다	워크플로우를 내보낼 때 비밀이 함께 나간다

3.2 실행 환경이 지켜야 하는 것

암호화 키

자격 증명을 암호화할 때 사용한다

암호화 키의 역할

자격 증명(credential)은 외부 시스템에 연결할 때 쓰는 정보(데이터베이스의 주소와 비밀번호, API의 키 등)를 워크플로우 밖에 따로 저장한 것이다(14쪽). n8n은 자격 증명을 데이터베이스에 저장할 때 이 키로 암호화하고, 워크플로우가 자격 증명을 쓸 때 같은 키로 다시 풀어 읽는다. 그래서 키가 없거나 바뀌면 저장해 둔 자격 증명을 열 수 없다.

암호화 키는 언제 생성되는가

암호화에 쓰는 키는 처음 실행할 때 자동으로 만들어져 n8n의 설정 폴더에 저장된다. 환경 변수로 키를 직접 지정할 수도 있다.

직접 지정하는 이유

자격 증명은 데이터베이스에, 키는 설정 폴더에 있으므로 둘 중 하나만 옮기면 자격 증명을 열 수 없다.

키를 환경 변수로 관리하면 **무엇을 함께 옮겨야 하는지가 눈에 보인다.**

주의할 점

키는 암호다. 워크플로우 파일이나 공유 문서에 적어 두면 안 된다.

실습에서는 값을 별도 파일(.env)에 두고 그 파일을 공유하지 않는다.

3.2 실행 환경이 지켜야 하는 것

시간대 설정

n8n이 실행을 시작하는 시각과 기록하는 시각의 기준을 한국 시간으로 맞춘다

시간대 설정이 중요한 이유

워크플로우는 시각을 두 가지로 쓴다. 정해진 시각에 실행을 시작하고(예: 매일 9시), 지금 시각을 값으로 기록한다(예: 문의 접수 시각).

"9시"와 "지금"이 어느 나라의 시각인지는 n8n에 설정한 시간대가 정한다. **그래서 n8n이 어느 시간대를 기준으로 할지 먼저 설정해야 한다.**

맞추지 않으면 생기는 문제

사람이 생각한 시각과 n8n이 실행하거나 저장한 시각이 달라진다.
시간대를 정하지 않으면 n8n은 기본으로 미국 뉴욕 시간대를 쓴다.

예: 한국 시각 오전 8시에 접수한 요청이 전날 오후 6시로 저장되어 승인 기한이 하루 앞당겨진다.

예: "매일 밤 9시 정리"가 한국 시각 다음 날 오전 11시에 실행된다.

설정하는 방법: n8n이 기준으로 쓸 시간대를 Asia/Seoul로 지정한다

환경 변수	언제 적용되는가	지정하지 않으면
GENERIC_TIMEZONE	일정 트리거가 실행 시각을 계산할 때, $now로 지금 시각을 읽을 때	미국 뉴욕 시간대
TZ	컨테이너 안의 프로그램이 로그처럼 운영 체제의 시각을 쓸 때	협정 세계시(UTC)

TZ도 같은 값으로 지정해 두어야 로그 같은 운영 체제의 시각이 같은 기준으로 남는다.

3.2 실행 환경이 지켜야 하는 것

자격 증명

연결 정보를 노드 설정 칸에 직접 적지 않고 워크플로우 밖에 따로 저장한다

자격 증명이란

외부 시스템에 연결할 때 쓰는 정보를 워크플로우 밖에 따로 저장한 것이다.

예: 데이터베이스의 주소와 비밀번호, API의 키

분리하는 두 이유

① 워크플로우를 파일로 내보내거나 다른 사람과 공유해도 비밀이 함께 나가지 않는다.

② 같은 연결을 여러 워크플로우가 함께 쓸 수 있고, 비밀번호가 바뀌면 한 곳만 고치면 된다.

3.3 노드 사이를 흐르는 데이터: 아이템

아이템은 배열로 흐른다

노드 사이를 흐르는 데이터를 아이템이라고 불렀다. 아이템은 하나씩이 아니라 **배열**로 흐른다

개인 가계부 워크플로우에서 지출 두 건이 흐르는 모습

```
[
  { "json": { "used_at": "2026-09-01",
              "merchant": "한빛분식",
              "amount": 12000 } },
  { "json": { "used_at": "2026-09-02",
              "merchant": "모두문구",
              "amount": 8000 } }
]
```

노드의 출력은 언제나 **아이템 여러 개가 들어 있는 배열**이다.
아이템이 한 건뿐일 때도 원소가 하나인 배열이다.

각 아이템은 **json 키** 아래에 실제 데이터를 담는다.

2장에서 데이터 계약으로 정한 것이 바로 이 json 키 아래의 구조다.
계약의 필드가 여기 그대로 들어온다.

파일을 다루는 아이템은 json 대신 또는 함께 다른 키를 쓴다. 파일 처리는 이 장에서 다루지 않는다.

3.3 노드 사이를 흐르는 데이터: 아이템

노드는 아이템을 하나씩 처리한다

배열을 받으면 아이템을 하나씩 꺼내 같은 설정을 적용한다

예: 지출 두 건이 **"금액이 10,000원을 넘는가"**를 검사하는 노드에 들어가면, 노드는 첫 아이템(12,000원)을 보고 참으로, 둘째 아이템(8,000원)을 보고 거짓으로 판정한다.

설계에 미치는 두 영향

한 건을 기준으로 그린 흐름이 여러 건에도 그대로 적용된다

사진 스무 장의 이름을 규칙에 맞게 바꾸는 개인 작업이든, 지원자 마흔 명에게 같은 형식의 안내를 보내는 업무든 흐름은 한 건 기준으로 그린다.

여러 건을 함께 보아야 하는 판단은 따로 설계한다

"이번 달 지출 합계가 예산을 넘었는가"는 아이템 하나만 보고 답할 수 없다.

여러 아이템을 하나로 모으는 단계를 따로 두어야 한다.

아이템 수가 바뀌는 지점

노드가 하는 일에 따라 출력 아이템 수는 **입력과 같거나, 줄거나, 늘거나, 하나로 모인다**

노드가 하는 일	출력 아이템 수	입력 3건일 때	입력 3건일 때의 예	언제 쓰는가
필드의 값을 바꾸거나 새 필드를 추가한다	입력 수와 같다	3	지출 3건의 "12,000원" 형식의 금액을 숫자로(12000) 바꾼다 → 3건	계약 형식으로 값을 맞출 때
조건에 맞는 것만 통과시킨다	0부터 입력 수까지	0~3	금액이 10,000원을 넘는 지출만 통과시킨다 → 2건	볼 필요가 없는 아이템을 걸러낼 때
조건에 따라 경로를 나눈다	경로마다 나뉘고, 합은 입력 수와 같다	합계 3	문의 3건을 교환과 취소로 나눈다 → 교환 2건, 취소 1건	유형별로 다르게 처리할 때
아이템 안의 목록을 아이템으로 나눈다	목록의 값들 개수의 합(늘거나 줄 수 있다)	목록의 갑들 개수	기사가 4건, 2건, 0건 든 응답 3건을 나눈다 → 6건	한 응답에 여러 건이 들어 있을 때
여러 아이템을 하나로 모은다	항상 1	1	지출 3건을 모아 합계를 낸다 → 1건	합계를 내거나 한 통으로 보낼 때

출력 아이템 수를 예상하지 못하면 다음 노드가 무엇을 받을지도 모른다. 기사 열 건이 목록 하나로 왔다면 아이템은 한 건이므로, "제목에 키워드가 있는가"를 검사하면 한 건 전체가 통과하거나 전부 걸러진다. 기사별로 판단하려면 목록을 아이템 열 건으로 나누는 단계를 먼저 둔다.

3.3 노드 사이를 흐르는 데이터: 아이템

중첩된 값과 아이템 나누기

나눌지 말지는 그 값 하나하나가 따로 처리되어야 하는지로 판단한다

나누지 않는다: 아이템 안의 객체

```
{
  "json": {
    "request_id": "REQ-0001",
    "order": {
      "product_name": "린넨 셔츠",
      "amount": 39000
    }
  }
}
```

2장에서 요청 기록의 주문 정보를 묶은 order 객체는 아이템 한 건 안의 값일 뿐 별도의 아이템이 아니다. 안의 값을 쓰려면 그 안으로 들어가면 된다. (${{ json.order.amount }})

나눈다: 따로 처리할 목록

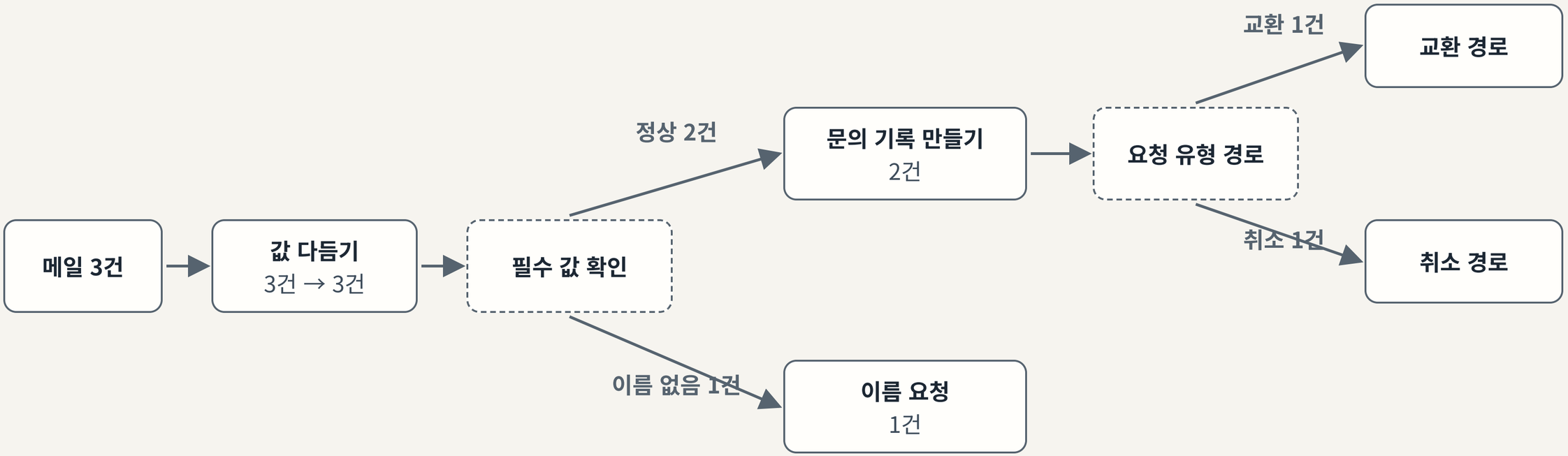
```
{
  "json": {
    "articles": [
      { "title": "기사 1", … },
      { "title": "기사 2", … }
      … 기사 10건 …
    ]
  }
}
```

한 아이템 안에 기사 열 건이 배열로 들어 있고 기사마다 다른 처리를 해야 한다면, 배열을 아이템으로 나눈다.

3.3 노드 사이를 흐르는 데이터: 아이템

문의 세 건이 분기를 지나는 모습

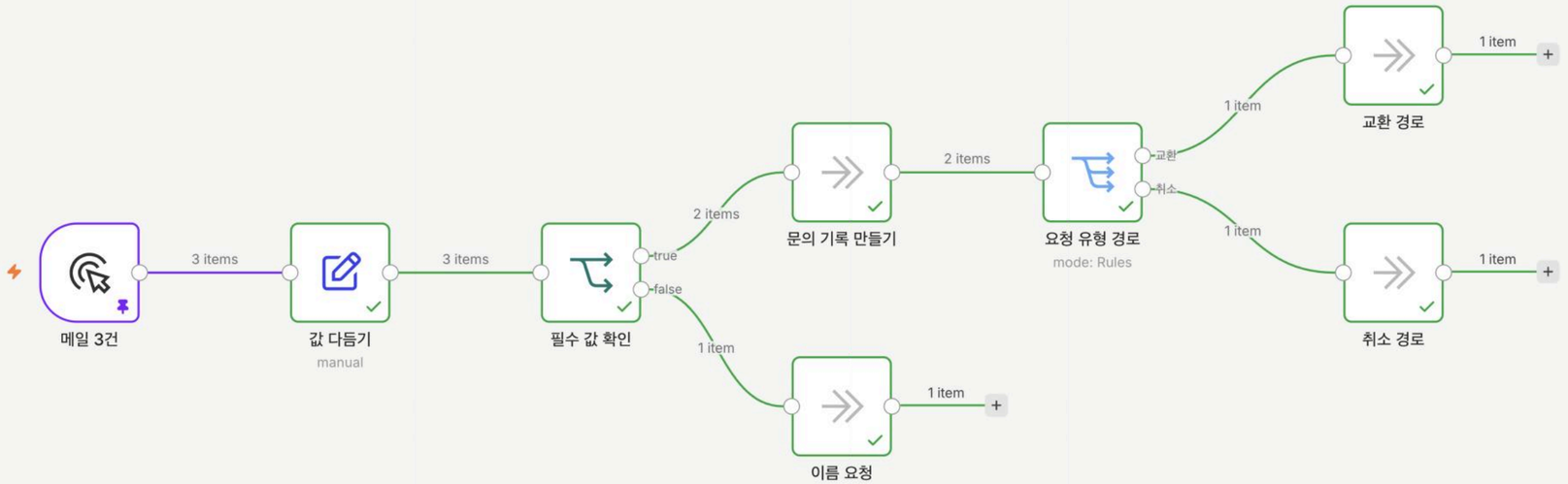
고객지원 메일함에 새 메일이 세 통 들어와 있으면, 메일 세 통은 아이템 세 건으로 흐른다 (하루마켓에 적용하기)



나뉜 뒤에도 각 경로는 자기에게 온 아이템만 처리한다. 그래서 문의가 한 통이든 서른 통이든 같은 워크플로우가 그대로 동작한다.

n8n 화면에서 아이템 수 읽기

앞 쪽의 흐름을 n8n에서 실행한 화면(execution)이다. 연결선마다 지나간 아이템 수가 표시된다 (하루마켓에 적용하기)



연결선의 아이템 수

실행한 뒤 연결선마다 지나간 아이템 수(3 items, 2 items, 1 item)가 보인다.

분기의 출력

If 노드는 true와 false로, Switch 노드는 규칙마다 출력이 나뉜다.

나뉜 수의 합

나뉜 경로의 수를 더하면 들어온 수와 같다(2 + 1 = 3, 1 + 1 = 2).

3.4 필드 참조와 표현식

고정값과 표현식

노드의 설정 칸에는 언제나 같은 고정값이나, 아이템마다 달라지는 값을 계산하는 표현식을 적는다



고정값

언제나 같은 값이다. 설정 칸에 그대로 적는다.

표현식

아이템마다 달라지는 값을 계산한다. {{ }} 안에 적는다. 화면의 초록 글씨가 표현식이다.

$json.필드명

{{ $json.order_id }}는 앞 노드가 전달한 아이템 가운데 **지금 처리 중인 아이템**의 order_id 값을 가리킨다.

아이템마다 다시 계산

표현식은 한 번 계산되어 고정되지 않는다. 아이템이 세 건이면 세 번 계산되어 세 값이 나온다.

3.4 필드 참조와 표현식

표현식에서 자주 쓰는 참조

$로 시작하는 이름으로 지금 아이템, 다른 노드, 실행 시점의 값을 가리킨다

쓰는 것	가리키는 값	언제 사용하나
$json	지금 처리 중인 아이템의 데이터	바로 앞 노드가 넘긴 아이템을 쓸 때
$json.필드명	그 아이템의 특정 필드	대부분의 경우
$('노드 이름')	이름으로 지정한 다른 노드의 출력	바로 앞이 아닌 앞선 노드의 값이 필요할 때
$now	지금 일시	접수 시각처럼 실행 시점을 기록·확인할 때
$today	오늘 0시	날짜 단위로 비교할 때
$execution	지금 실행에 관한 정보	실행 식별자를 기록에 남길 때
$workflow	이 워크플로우에 관한 정보	어느 워크플로우의 결과인지 남길 때

중첩된 값은 점을 이어 적는다: {{ $json.order.amount }}는 order 객체 안의 amount

필드의 값이 배열일 때 그 배열 원소는 0부터 번호(index)로 적는다: {{ $json.articles[0].title }}은 첫 번째 기사의 제목

3.4 필드 참조와 표현식

값이 없는 세 가지 경우

값이 없는 데이터는 **null, 빈 문자열, 필드 없음**의 세 모습으로 들어오고, 모습마다 표현식이 읽는 값과 조건 검사의 결과가 다르다

2장에서 데이터 계약을 정할 때 빈 문자열, null, 필드가 없는 경우를 구분해야 한다고 했다. 그 이유가 실행할 때 여기서 드러난다.

경우	데이터의 모습	표현식으로 읽으면	주의할 점
값이 null이다	"order_id": null	null이 나온다	계약이 정한 올바른 표기다. "값이 비어 있는가" 검사에서 비어 있다고 판정된다
값이 빈 문자열이다	"order_id": ""	빈 문자열 ""이 나온다	"값이 있다"로 잘못 판정될 수 있다
필드 자체가 없다	order_id 키가 없다	정해지지 않은 값(undefined)이 나온다	이 값에 문자열 기능을 써도 노드가 멈추지 않고 빈 값이 나온다

이 절에서 가장 많은 오류가 나오는 지점이다. 특히 셋째 경우를 조심해야 한다(다음 쪽).

3.4 필드 참조와 표현식

필드가 없을 때 조용히 넘어가는 값

필드 자체가 없는 값에 어떤 기능을 쓰느냐에 따라 결과가 다르다

문자열 기능: 조용히 빈 값이 된다

없는 필드에 앞뒤 공백을 없애는 기능 같은 문자열 기능을 쓰면, 워크플로우를 실행해도 오류가 나지 않고 그 칸만 빈 값이 된다.

실행은 성공으로 기록되므로 값이 없다는 사실을 아무도 알려 주지 않고, 빈 값이 다음 단계로 그대로 넘어간다.

숫자·날짜로 바꾸는 기능: 오류로 멈춘다

n8n이 표현식에 따로 제공하는 변환 기능 가운데 문자열을 숫자나 날짜로 바꾸는 기능(toNumber(), toDateTime())은 같은 상황에서 오류로 멈춘다.

같은 "값 없음"이 쓰는 기능에 따라 조용히 넘어가기도 하고 멈추기도 한다. 그러므로 기능의 동작에 기대지 말고 값이 없는 경우를 직접 다루어야 한다(방법은 다음 쪽).

3.4 필드 참조와 표현식

계약의 표기로 먼저 통일한 다음 검사한다

세 경우가 다르게 다뤄지므로, 표기를 통일하는 단계가 조건 검사보다 앞에 온다

1 **계약의 표기로 통일한다**

웹 양식은 빈 칸을 빈 문자열로 보내고, 어떤 양식은 아예 필드를 빼고 보낸다. 이 둘을 null로 통일해 두면 그다음 단계의 조건은 한 가지만 검사하면 된다.

2 **$ifEmpty로 감싼다**

{{ $ifEmpty($json.body.order_no, null) }}은 **값이 비어 있거나 없으면 null을, 있으면 그 값을 낸다**. 필드가 없을 때도 빈 값 대신 정해 둔 값으로 다음 기능을 이어 쓸 수 있다.

3 **분기할 때는 조건 연산을 쓴다**

조건 분기 노드에는 "값이 비어 있는가", "필드가 있는가" 같은 연산이 데이터 타입별로 준비되어 있어, 필드가 없는 아이템이 들어와도 노드가 멈추지 않고 판정한다.

데이터 타입을 계약에 맞추기

값을 계약이 정한 데이터 타입과 형식으로 바꾸는 일이 여기서 일어난다

금액: 문자열 기능을 이어 쓴다

2장에서 금액을 "39,000원" 같은 문자열로 두면 계산과 비교가 되지 않는다고 했다.

쉼표와 단위를 떼고 숫자로 바꾸려면 앞뒤 공백 없애기, 찾은 문자를 모두 바꾸기, 숫자로 바꾸기 같은 문자열 기능을 이어 쓴다. 이어 쓰면 한 줄로 정리된다.

```
{{ $json.price.trim().replaceAll(",", "")
   .replaceAll("원", "").toNumber() }}
"39,000원" → 39000 (숫자)
```

날짜: 입력 형식을 알려 준다

문자열을 날짜로 바꾸는 기능은 널리 쓰이는 몇 가지 표기 방식만 알아본다. 사람이 손으로 적은 2026/09/03 12:40 같은 표기는 알아보지 못할 수 있다.

도구가 알아보지 못하는 표기는 입력 형식을 직접 알려 주어야 한다. 날짜를 다루는 전용 노드(Date & Time)에 입력 형식을 지정하는 설정이 있다.

값이 어떤 표기로 들어올지 모르는 채로 변환을 맡기면, 어떤 행은 변환되고 어떤 행은 변환되지 않거나 엉뚱한 날짜가 된다. 2장에서 계약에 형식을 적어 둔 이유가 여기 있다.

3.4 필드 참조와 표현식

값을 바꾸는 세 가지 방법

같은 변환을 표현식, 전용 노드, 코드로 적을 수 있다

방법	어떤 때 쓰는가	한계
표현식	필드 하나의 값에 대해 한 줄(구문)로 바꿀 때 (예: "39,000원" → 39000, 앞뒤 공백 없애기)	규칙이 복잡해서 값에 따라 처리가 여러 번 갈리면(분기) 한 줄로는 읽기 어려워진다
전용 노드	목록을 나누거나 모으거나 날짜를 다룰 때처럼 정해진 작업	준비된 작업의 범위 안에서만 쓴다
코드	규칙이 복잡하고 여러 단계를 거칠 때	화면에서 흐름이 보이지 않고, 읽고 고칠 사람이 코드를 알아야 한다

고르는 기준은 1장의 "단순한 방식부터 고른다"와 같다. 표현식으로 읽히면 표현식으로 두고, 전용 노드가 있으면 전용 노드를 쓰고, 둘 다 어려워질 때 코드로 옮긴다. **처음부터 코드로 가면 노코드 도구를 쓰는 이유가 줄어든다.**

3.4 필드 참조와 표현식

웹 문의에서 문의 기록의 값 만들기

웹 양식이 보내는 칸 이름과 2장 문의 기록 계약의 필드명이 다르므로 여기서 맞춘다 (하루마켓에 적용하기)

웹 양식이 보내는 값	계약 필드	맞추는 방법
body.name	**customer_name**	이름을 바꾸고 앞뒤 공백을 없앤다
body.order_no	**order_id**	이름을 바꾸고, 비어 있으면 null, 있으면 공백을 없앤 값
body.content	**message**	이름을 바꾸고 앞뒤 공백을 없앤다
body.agree_privacy	(옮기지 않는다)	양식은 동의 칸에 체크해야 보내지므로 들어온 문의는 모두 동의한 문의다. 계약에 없는 값이라 맞추는 과정에서 버린다
(보내지 않는다)	**received_at**	워크플로우가 지금 시각을 ISO 8601로 만든다
(보내지 않는다)	**channel**	웹 문의를 받는 워크플로우이므로 web으로 채운다

inquiry_id는 이 표에 없다. 중복되지 않는 번호를 발급하려면 번호를 관리하는 곳이 있어야 하고, 그 자리는 업무 데이터베이스다. 값을 만들 수 없을 때 아무 값이나 넣는 것은 계약을 지키는 것이 아니다.

3.5 실행 결과 읽기

수동 실행과 프로덕션 실행

편집 화면에서 고친 내용은 자동으로 저장되지만, 저장만으로 프로덕션에 반영되지는 않는다

편집
편집 화면에서 노드와 설정을 고친다.

→

자동 저장
고친 내용이 저장된다. 프로덕션에는 아직 반영되지 않는다.

→

Publish
저장된 버전을 Publish 버튼으로 올린다.

→

프로덕션 실행
올린 버전을 그대로 쓴다. 그 뒤의 편집은 다시 Publish하기 전까지 반영되지 않는다.

구분	수동 실행	프로덕션 실행
언제 일어나는가	편집 화면에서 직접 시작할 때	Publish한 워크플로우의 트리거가 시작할 때
결과를 보는 곳	편집 화면의 노드별 입력·출력 패널	실행 목록에서 지난 실행을 열어 본다
무엇에 쓰는가	만드는 중에 값을 확인하며 고친다	실제로 동작한 결과를 확인한다

만드는 동안에는 수동 실행으로 값을 보며 고치고, 다 만든 뒤에 Publish해 프로덕션 실행으로 넘긴다.
프로덕션에서 내릴 때는 Publish 옆의 메뉴에서 Unpublish를 고른다.

3.5 실행 결과 읽기

값이 처음 틀어진 곳 찾기

값이 한 번 틀어지면 뒤의 모든 노드가 틀린 값을 받으므로, 마지막 노드부터가 아니라 앞에서부터 본다

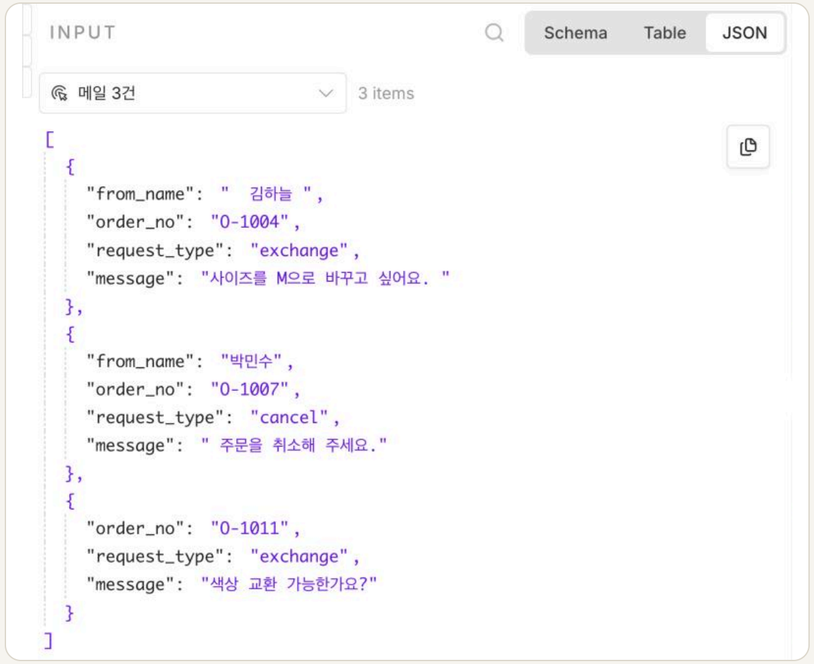
순서	확인할 것	무엇을 알 수 있는가
1	트리거 노드의 출력	애초에 들어온 데이터가 예상과 같은가
2	각 노드의 입력과 출력을 앞에서부터 비교	어느 노드에서 값이 처음 달라졌는가
3	그 노드의 설정과 표현식	참조한 필드 이름이 맞는가, 값이 없을 때를 처리했는가
4	그 노드를 지난 아이템 수	걸러지거나 나뉘면서 예상과 달라지지 않았는가

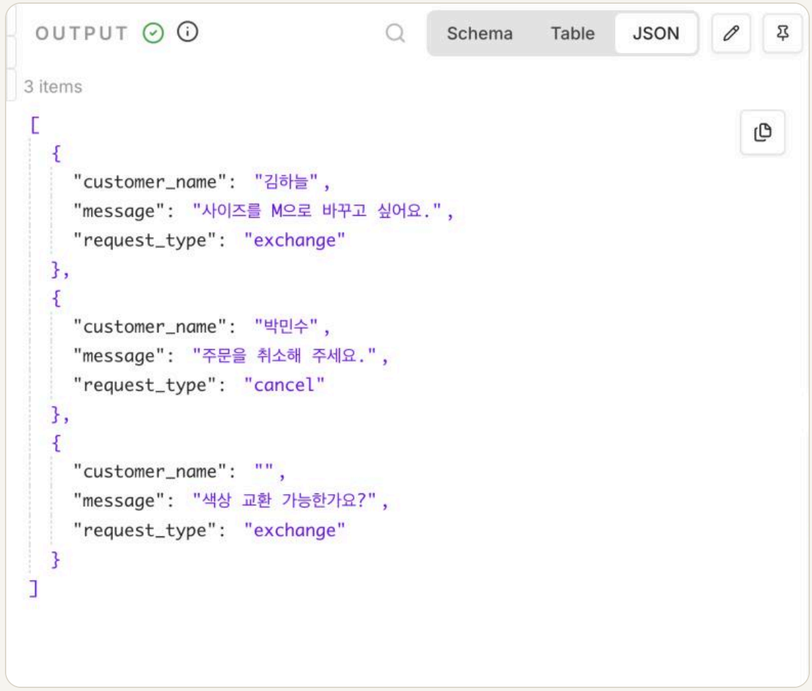
오류로 멈춘 실행에서는 멈춘 노드, 그 노드가 받은 입력, 오류 메시지를 함께 본다. 메시지만 읽고 설정을 고치면 원인을 지나치기 쉽다. 값이 없는 필드에 문자열 기능을 쓴 경우라면, 고쳐야 하는 곳은 그 노드가 아니라 **값을 통일하지 않고 넘긴 앞 단계**일 수 있다.

3.5 실행 결과 읽기

노드의 입력과 출력 비교하기

노드 상세 화면에서 이 노드가 받은 값과 낸 값을 나란히 놓고 본다





입력 패널

이 노드가 받은 아이템이다. 앞 노드의 출력과 같다.

출력 패널

이 노드가 낸 아이템이다. 패널 위쪽에 아이템 수(3 items)도 보인다.

이 화면에서 읽을 것

셋째 아이템은 입력에 from_name이 없는데, 출력의 customer_name은 빈 문자열이고 실행은 성공했다.

3.5 실행 결과 읽기

실행 식별자로 요청 기록과 실행 기록 잇기

업무 진행은 요청 기록으로 답하고, 그 값을 만든 실행은 요청 기록에 적어 둔 실행 식별자로 찾아간다

① 업무 진행은 실행 기록으로 답하지 않는다

두 기록은 역할이 다르다.

- **실행 기록:** 워크플로우가 어떻게 동작했는지의 기록
- **요청 기록:** 업무가 지금 어디까지 진행 됐는지 답하는 기록

실행 기록은 보관 기간이 지나면 정리될 수 있고, 한 요청이 여러 실행에 걸쳐 진행된다. 그래서 실행 기록으로 업무 진행을 답하면 지난 실행들을 뒤져 추측해야 한다(10쪽 표의 마지막 줄).

② 두 기록은 실행 식별자로 잇는다

실행할 때마다 각 실행에는 **실행 식별자**(execution id)가 붙고, 표현식에서 $execution으로 읽는다.

요청 기록에 처리 결과를 남길 때 실행 식별자를 함께 적어 두면, 요청 기록의 값이 이상할 때 그 값을 만든 실행을 찾아가 원인을 읽을 수 있다(30쪽). 2장 기록 항목에서 "3장 이후 채운다"고 비워 둔 칸이 이 값이다.

3.5 실행 결과 읽기

데이터 고정(Pin)

만드는 중에는 같은 입력으로 여러 번 실행하며 고친다

데이터 고정(data pinning)이란

노드의 출력 데이터를 고정해 두고 다음 실행에서 그 값을 쓰는 기능이다.

트리거가 외부에서 오는 요청이어도, 고칠 때마다 요청을 다시 보내지 않아도 된다.

고정한 데이터 고치기

고정한 데이터는 편집 화면에서 직접 고칠 수 있다.

값이 없는 경우나 형식이 틀린 경우 같은 상황을 실제로 만들지 않고도 시험할 수 있다.

고정한 데이터는 편집 중에만 쓰인다. Publish한 워크플로우의 프로덕션 실행은 고정 데이터를 무시하고 실제로 들어온 값을 쓴다. 편집 화면에서는 잘 되는데 실제로는 다르게 동작한다면, 고정 데이터가 남아 있지 않은지 확인한다.

3.5 실행 결과 읽기

customer_name이 비어 있을 때 예

웹 문의를 정규화한 결과에서 customer_name이 비어 있면.
앞에서부터 보면 원인이 셋 중 하나로 좁혀진다

① 트리거 출력에 이름 칸이 없었다

양식이 이름 칸을 아예 보내지 않았다.

② 참조한 필드 이름이 달랐다

값을 다듬는 노드가 참조한 필드 이름이
양식이 보낸 이름과 달랐다.

③ 빈 문자열을 통과시켰다

조건 검사가 빈 문자열을 "값이 있다"로
판정해 통과시켰다.

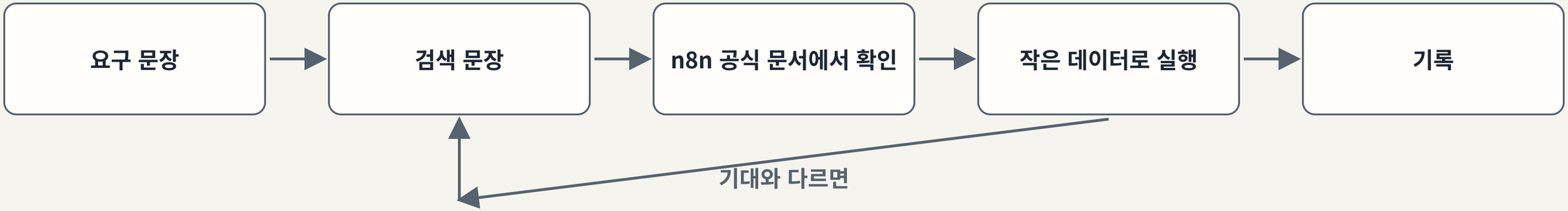
세 경우 모두 트리거 출력 한 번, 다듬은 결과 한 번을 보면 어느 쪽인지 드러난다.

3.6 요구에서 기능을 찾아 연결하기

기능 탐색 절차

노드 이름을 외우지 않고, 워크플로우가 해야 할 일(요구)에서 출발해 기능을 찾는다

n8n에는 수백 개의 노드가 있고 버전마다 늘어난다. 이름을 외워서 쓰는 방식은 오래가지 않는다. 대신 요구에서 출발해 기능을 찾는 절차를 익히면 처음 보는 연동에도 같은 방법을 쓸 수 있다.



요구 문장(업무의 말)
예1: 매주 금요일 오후에 시작하게 하고 싶다
예2: 담당자에게 메신저로 알리고 싶다

검색 문장(기능의 말)
예1: 일정에 따라 워크플로우를 시작하는 트리거
예2: 외부 서비스로 메시지를 보내는 기능

문서에서 찾은 노드는 후보일 뿐이다. 문서를 읽은 것만으로는 그 기능을 고른 것이 아니며, 작은 데이터를 넣어 출력을 본 뒤에야 쓸 수 있다고 판단한다.

3.6 요구에서 기능을 찾아 연결하기

기능의 네 갈래

데이터가 어디에서 와서 어디로 가는가로 기능을 나눈다

트리거

언제 시작하는가

흐름을 시작한다.

외부 연결

밖에서 가져오거나 밖으로 내보내는가

워크플로우 밖의 시스템과 데이터를 주고받는다.

변환

값을 어떻게 바꾸는가

이미 들어와 있는 값을 바꾼다.

흐름 제어

흐름을 어떻게 이어 가는가

값을 바꾸지 않고 아이템이 지나갈 길만 정한다.

1장에서 본 노드의 두 갈래(트리거 노드와 액션 노드) 중 액션 노드가 하는 일을 연결·변환·흐름 제어로 나눈 것이다. 처음 보는 요구도 이 넷 중 어디에 해당하는지 먼저 정하면 찾을 범위가 좁아진다.

요구 문장을 검색 문장으로 바꾸기 ①

"무엇을 하고 싶은가"를 "어떤 동작인가"로 바꾼다

요구 문장	검색 문장	찾게 되는 갈래
매주 금요일 오후에 시작하게 하고 싶다	일정에 따라 워크플로우를 시작하는 트리거	트리거
다른 시스템이 보내는 알림을 받고 싶다	외부 요청을 받는 트리거	트리거
사내 시스템에서 주문 정보를 가져오고 싶다	외부 API를 호출해 응답을 받는 기능	연결
담당자에게 메신저로 알리고 싶다	외부 서비스로 메시지를 보내는 기능	연결

업무 요구는 업무 용어로 적혀 있고 문서는 기능 용어로 적혀 있다. 둘 사이를 옮기는 것이 기능 탐색의 첫 번째 일이다.

요구 문장을 검색 문장으로 바꾸기 ②

흐름 제어와 변환 갈래의 예

요구 문장	검색 문장	찾게 되는 갈래
빈 칸이 있으면 다음으로 넘어가지 않게 하고 싶다	조건으로 아이템을 걸러 내는 기능	흐름 제어
종류별로 다르게 처리하고 싶다	여러 경로로 나누는 기능	흐름 제어
나뉘었던 두 경로의 결과를 한자리에 모으고 싶다	여러 입력을 하나로 합치는 기능	흐름 제어
답이 올 때까지 다음 단계를 미루고 싶다	실행을 멈췄다가 재개하는 기능	흐름 제어
응답 하나에 여러 건이 들어 있다	목록을 아이템으로 나누는 기능	변환
같은 항목이 여러 번 들어온다	중복을 없애는 기능	변환
날짜 표기를 통일하고 싶다	날짜 형식을 바꾸는 기능	변환

흐름 제어의 범위와 네 갈래 밖의 기능

이 장에서는 흐름 제어 가운데 분기만 쓴다

흐름 제어에 들어가는 것

조건으로 경로를 나누는 **분기** 말고도 다음이 흐름 제어에 들어간다.

나뉜 경로를 합치기, 여러 건을 나누어 반복 처리하기, 정해진 조건까지 기다리기, 다른 워크플로우를 불러 쓰기

분기 밖의 것은 필요한 장에서 다룬다.

네 갈래에 들어가지 않는 기능

모델에게 판단을 맡기는 AI 기능은 1장에서 정한 판단 주체와 함께 다룬다.

실행이 실패했을 때 알리고 되살리는 기능은 2장에서 설계한 실패 경로와 함께 다룬다.

두 가지는 기능을 찾는 문제이기 전에 설계 판단의 문제여서 따로 둔다.

공식 문서에서 확인할 네 가지

후보를 찾았으면 공식 문서에서 네 가지를 확인한다

확인할 것	왜 확인하는가
이 기능이 하는 일	이름만 보고 짐작한 것과 실제 동작이 다를 수 있다
필요한 인증	계정이나 키가 필요하면 준비 비용이 생긴다
입력과 출력	앞뒤 노드와 아이템 구조가 맞는지 판단한다
적용 버전에서의 이름	버전에 따라 이름과 설정 칸이 바뀐다

특히 입력과 출력은 아이템 수가 바뀌는 지점(17쪽)과 함께 본다. 아이템 수가 바뀌는 기능인지 알아야 다음 노드가 무엇을 받을지 예상할 수 있다.

3.6 요구에서 기능을 찾아 연결하기

생성형 AI가 알려 준 노드 이름

"n8n에서 이런 걸 하려면 어떤 노드를 쓰나요?"의 답은 출발점으로 쓸 만하지만 그대로 믿으면 안 된다

버전에 따라 이름과 설정이 달라진다

예전 이름으로 알려 주면 화면에서 찾을 수 없다.

존재하지 않는 이름을 그럴듯하게 알려 주기도 한다

있을 법한 이름이라 검증 없이 넘어가기 쉽다.

그래서 추천받은 이름도 후보로만 적고 같은 절차를 밟는다. 공식 문서에서 찾지 못했다면 그 사실 자체를 기록에 남긴다. 확인하지 않은 정보를 확정된 사실처럼 설계에 넣지 않는 것은 n8n에만 해당하지 않으며, 자동화를 다루는 내내 필요하다.

3.6 요구에서 기능을 찾아 연결하기

하루마켓 웹 문의의 요구를 검색 문장으로

웹 문의를 받아 필수 값을 검사하고 유형별로 나누는 세 요구가 모두 트리거와 흐름 제어 갈래에 들어간다 (하루마켓에 적용하기)

요구	검색 문장	갈래
웹 양식에서 보낸 문의를 받는다	웹 입력 양식을 통해 외부 요청을 받는 트리거	트리거
필수 값이 비어 있으면 다음으로 넘기지 않는다	값이 비어 있는지 검사해 경로를 나누는 기능	흐름 제어
요청 유형별로 경로를 나눈다	여러 경로로 나누는 기능	흐름 제어

연결과 변환 갈래가 없다는 점에서, 이 워크플로우가 값을 바꾸거나 밖에서 가져오기보다 받고 나누는 일에 가깝다는 것을 미리 알 수 있다. 밖에서 주문 정보를 가져오는 연결 갈래는 다음 장에서 이 경로의 끝에 붙인다.

3.7 정리

세 질문은 하나의 사슬로 이어진다

1

어디에서 실행하는가는 지켜야 할 다섯 가지로 답한다. 영속 저장, 두 데이터베이스의 구분, 암호화 키, 시간대, 자격 증명을 "없으면 어떤 일이 생기는가"로 이해해 둔다.

2

데이터가 어떤 모습으로 흐르는가는 아이템의 배열로 답한다. 아이템 수가 바뀌는 지점을 예상하고, 값이 없는 세 경우를 계약의 표기로 먼저 통일한 뒤 조건을 검사한다.

3

기능을 어떻게 찾는가는 검색 문장, 공식 문서의 네 가지 확인, 작은 데이터 실행, 기록의 절차로 답한다. 노드 이름은 이 절차의 결과이지 출발점이 아니다.

세 질문은 서로를 필요로 한다

환경의 시간대를 맞추지 않으면 $now가 만든 값이 계약을 어긴다.

아이템 구조를 모르면 실행 결과를 읽어도 무엇이 틀렸는지 알 수 없다.

결과를 읽지 못하면 새로 찾은 기능이 맞는지 판단할 수 없다.

다음 장: 이 장에서 나눈 경로의 끝에 외부 시스템을 조회하는 도구를 붙이고, 2장에서 정한 계약을 도구의 입출력 계약으로 넓힌다. 규칙만으로는 가릴 수 없었던 요청 유형의 분류도 다시 다룬다.